PROJECT DOCUMENTATION

Script-Controlled ACL – Restrict Record Access Based on Field Value

1. Introduction

This document describes the functional design and implementation of the Script-Controlled Access Control List (ACL) solution developed on ServiceNow to restrict record access based on a field value.

Project Title: Script-Controlled ACL – Restrict Record Access Based on Field Value

Team Members: Yegenesh M (Team Leader), Santhoshkumar U, Vishnuvarathan M, Selvakumar R.

Team ID: SWTID-2026-8089

2. Project Overview

Purpose: Implement script-controlled ACLs so that users can view, create, edit and delete records of the Institution Details table only when they hold the required role and the record meets the specified condition (Branch is EEE), while administrators retain full access.

Key Features:

- Custom table Institution Details (u_institution_details) with Branch choice field (ECE, EEE, CSE)
- Test user (EEE User) and custom roles bb1, bb2, bb3 and bb4
- Script-controlled READ ACL with role, data condition and advanced script
- Separate CREATE (bb2), WRITE (bb3) and DELETE (bb4) ACLs
- Administrator override through gs.hasRole('admin')
- Verification through user impersonation

3. Solution Architecture

The solution uses ServiceNow User Administration (users and roles), a custom table, and the Access Control (ACL) engine as the main components.

End-to-end process:

1. The administrator creates the EEE User and the custom roles bb1, bb2, bb3 and bb4.
2. The roles are assigned to the EEE User.
3. The custom table u_institution_details is created with the required fields and sample records for ECE, EEE and CSE branches.
4. The administrator elevates to the security_admin role and creates the READ, CREATE, WRITE and DELETE ACLs.

5 When a user opens the Institution Details list, ServiceNow evaluates the role requirement, data condition and ACL script.

6 If all conditions are met, the access is allowed; otherwise it is denied.

7 The behaviour is verified by impersonating the EEE User, a user without roles and the administrator.

4. ServiceNow Configuration

The implementation follows the project procedure supplied for the Script-Controlled ACL lab.

Table	Institution Details (u_institution_details)
Application	Global
Elevated role	security_admin (required to create ACLs)
ACL Type	record (Decision type: Allow If, Admin overrides enabled)
Operations	read, create, write, delete

5. ACL Configuration

The following Access Control rules are configured on the u_institution_details table:

ACL	Requires Role	Data Condition	Purpose
READ	bb1	Branch is EEE	View only EEE branch records (advanced script enabled)
CREATE	bb2	None	Enables the New button to create records
WRITE	bb3	None	Edit records
DELETE	bb4	None	Delete records

The READ ACL has the Advanced option enabled and uses the following script:

```
(function () {
  // Allow admin users full access
  if (gs.hasRole('admin')) {
    return true;
  }
  // Allow only EEE branch users to see EEE records
  if (gs.hasRole('bb1')) {
    return true;
  }
  // Deny access for all others
  return false;
})();
```

6. User, Role and Data Configuration

An EEE User is created with the email eeeuser@gmail.com and the custom roles bb1, bb2, bb3 and bb4 are created and assigned to this user. The Institution Details table contains the fields Student Roll Number (auto number), Student Name (Reference – User), Faculty Name (Reference – User), Branch (Choice – ECE, EEE, CSE), Email, Phone Number and Description. Sample records are created with Branch values ECE, EEE and CSE to test record-level filtering.

7. Workflow / Process Flow

Admin creates users, roles and ACLs → User opens Institution Details list → ACL checks role and Branch condition → Allow (matching records shown) or Deny (no records / access blocked)

The implementation procedure verifies the result by impersonating the EEE User and opening the list u_institution_details.list, then impersonating a user without the roles and finally the administrator.

8. Testing

Testing includes User Acceptance Testing and execution observations. The UAT test cases verify read, create, write and delete access for each role, together with the denial of access for users without roles.

No.	Test Case	Expected Result	Result
1	EEE User with bb1 opens the Institution Details list	Only EEE branch records are displayed	Pass
2	User without bb1–bb4 roles opens the list	Access is denied / no records visible	Pass
3	Administrator opens the list	All records (ECE, EEE, CSE) are displayed	Pass
4	User with bb1 and bb2	EEE records visible with the New button	Pass
5	User with bb1, bb2 and bb3 edits the Description	Record is updated successfully	Pass
6	User with bb1–bb4 deletes an EEE record	Record is deleted after confirmation	Pass

Quantitative response-time, throughput and concurrency measurements were not captured in the available project evidence and are therefore not reported as measured values.

9. Screenshots / Demo

The following evidence is included from the project implementation:

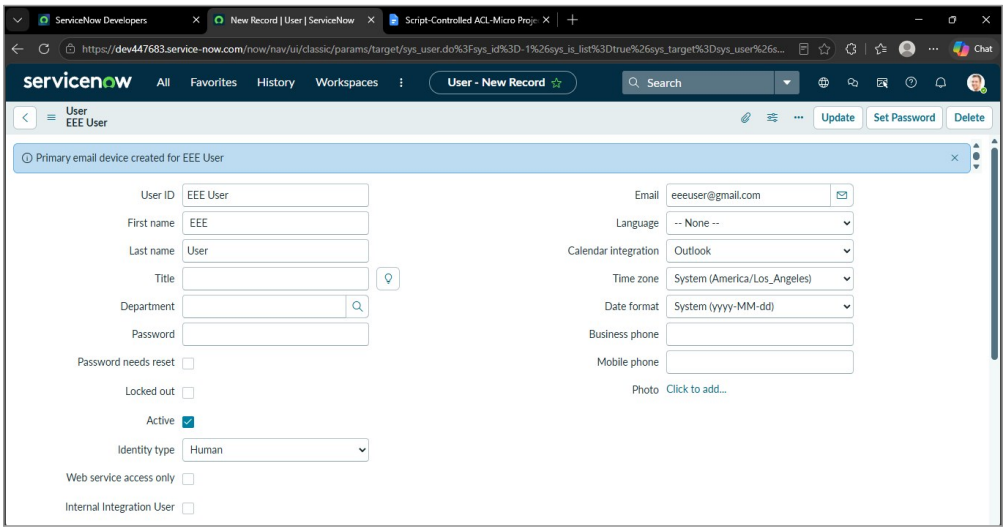


Figure 1: EEE User created (User Administration → Users)

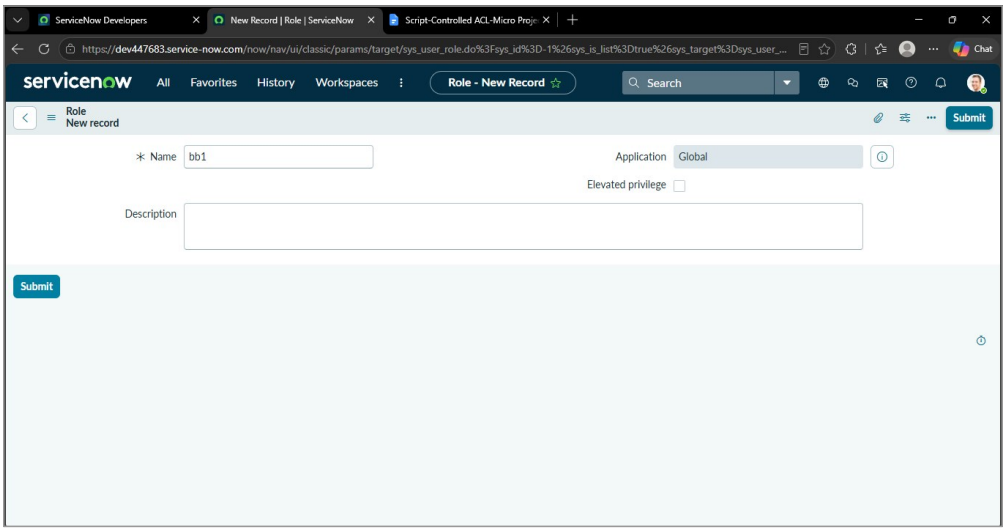


Figure 2: Creation of custom role bb1 (bb2, bb3 and bb4 created in the same way)

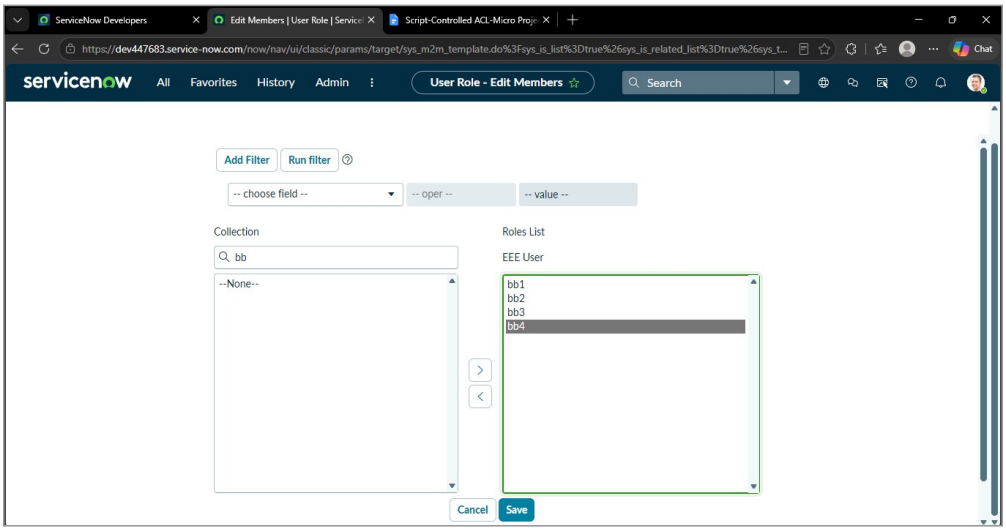


Figure 3: Roles bb1–bb4 assigned to the EEE User

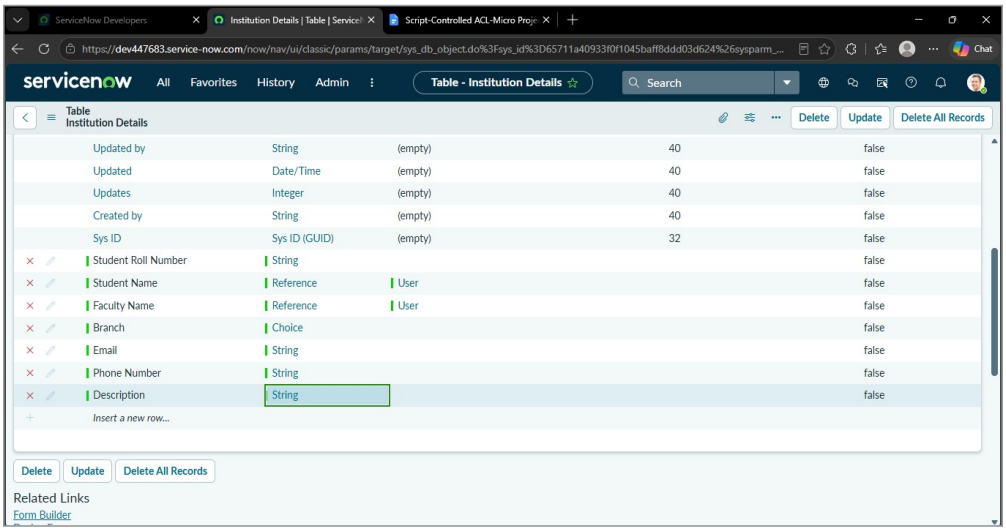


Figure 4: Institution Details table (u_institution_details) with required fields

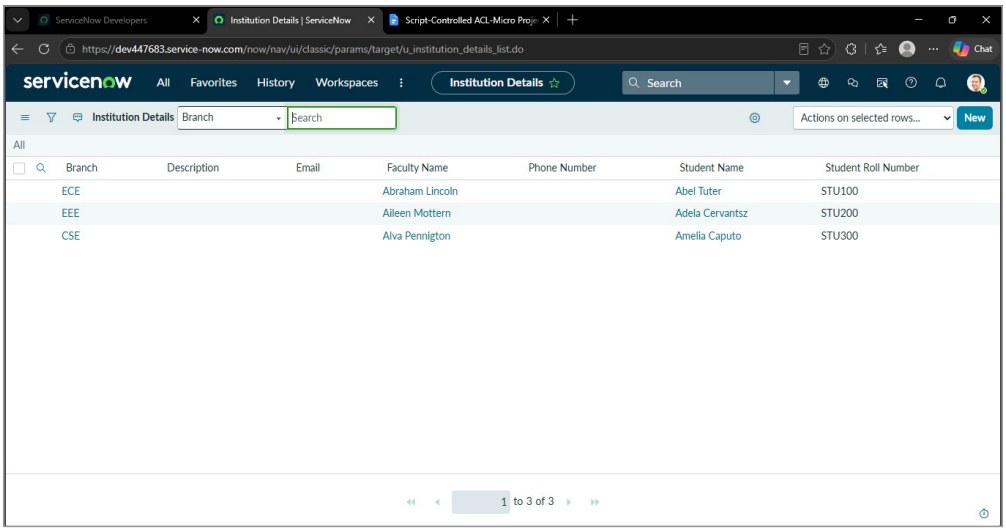


Figure 5: Sample records with Branch values ECE, EEE and CSE (admin view)

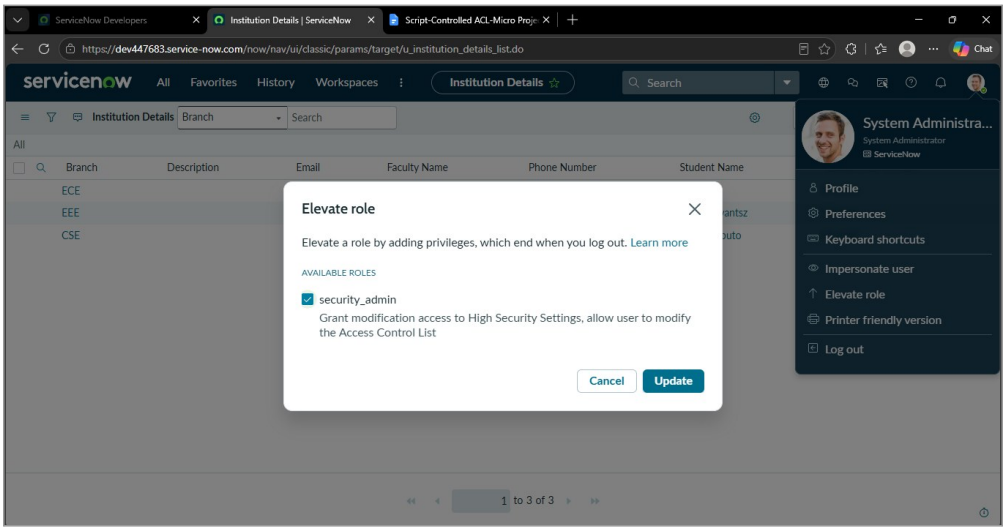


Figure 6: Elevating the security_admin role to create ACLs

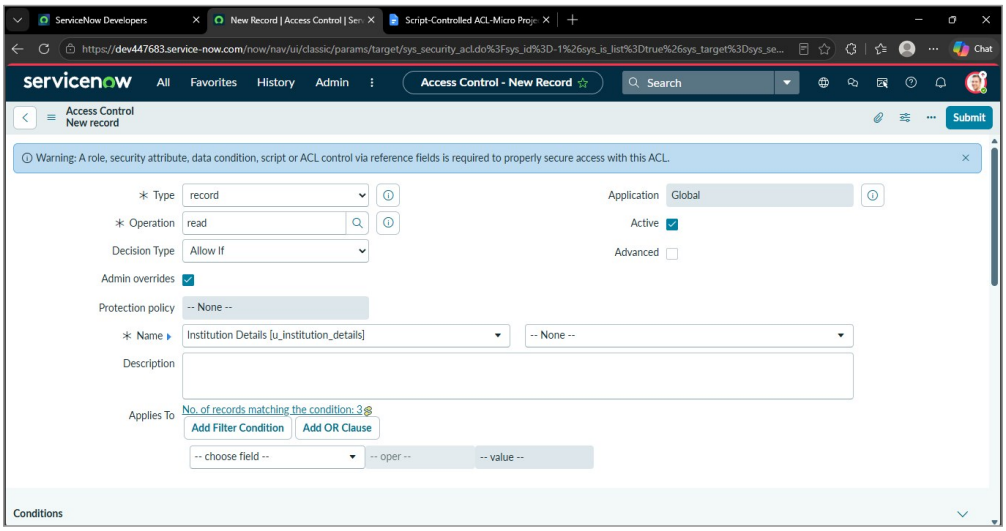


Figure 7: Read ACL – Type record, Operation read, Name u_institution_details

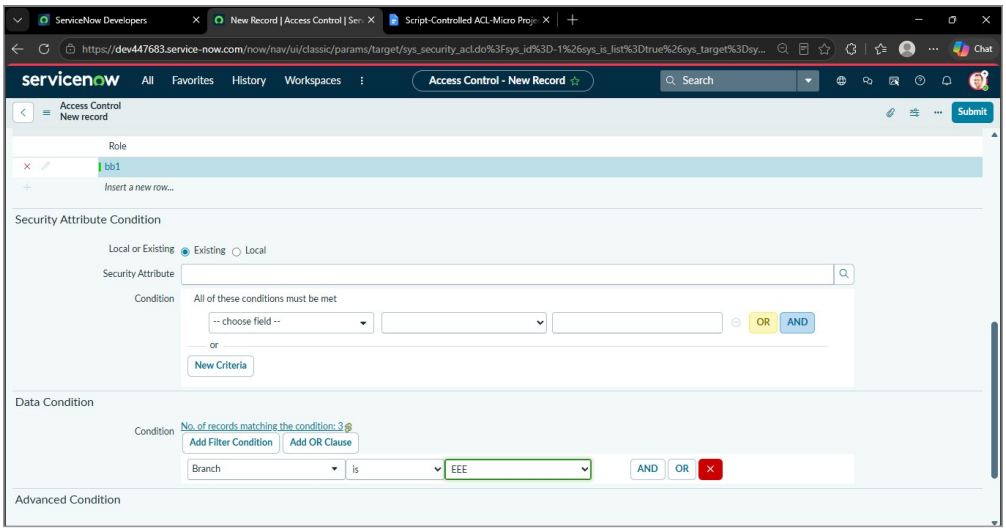


Figure 8: Read ACL – Requires role bb1 and Data condition Branch is EEE

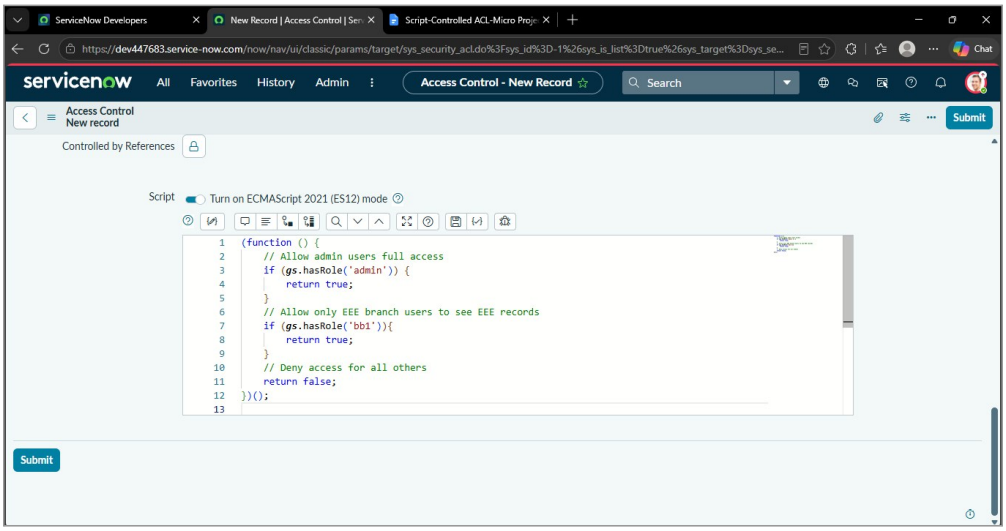


Figure 9: Read ACL – Advanced script using gs.hasRole()

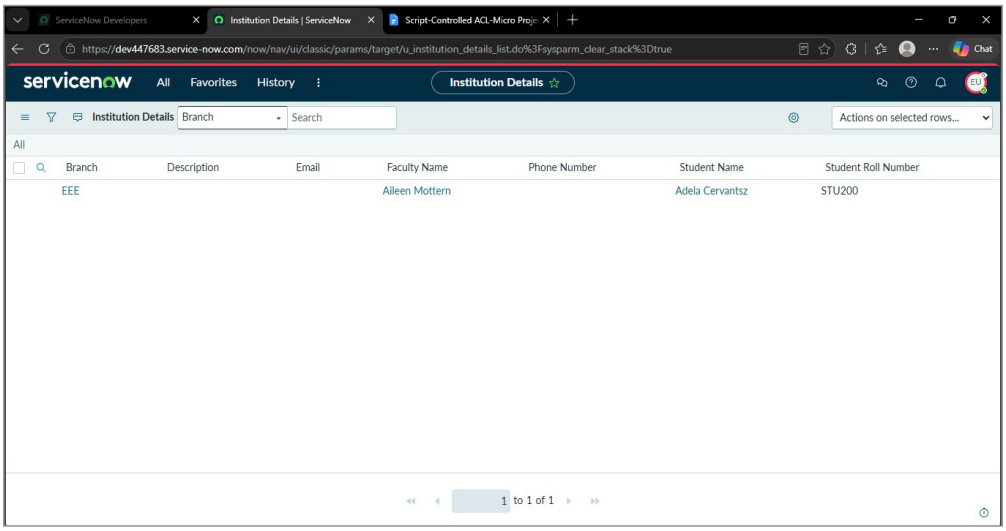


Figure 10: EEE User (bb1) can view only EEE branch records

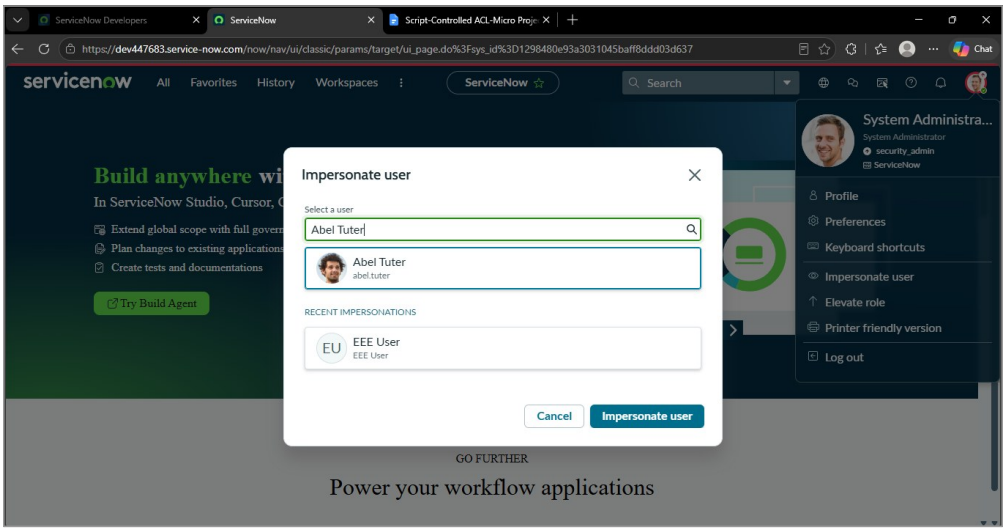


Figure 11: Impersonating a user without the custom roles

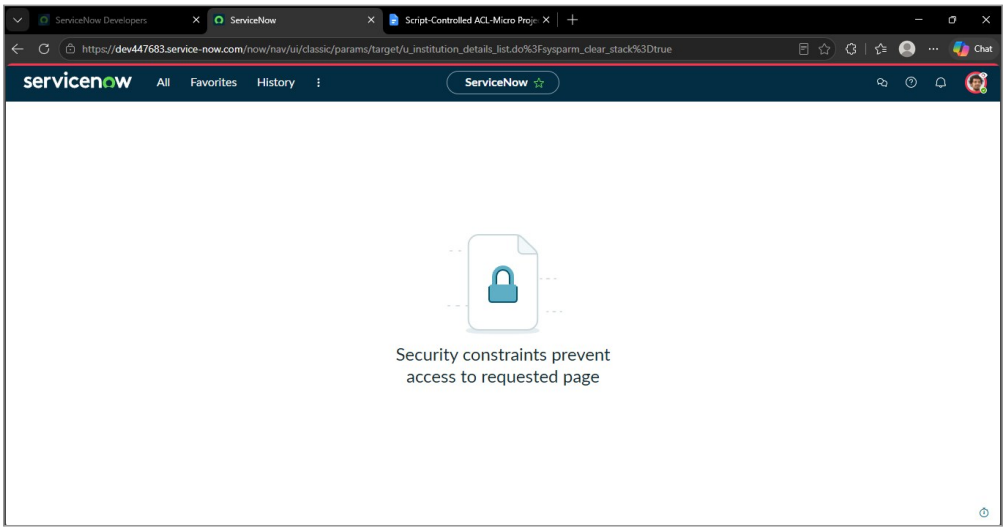


Figure 12: User without roles – security constraints prevent access

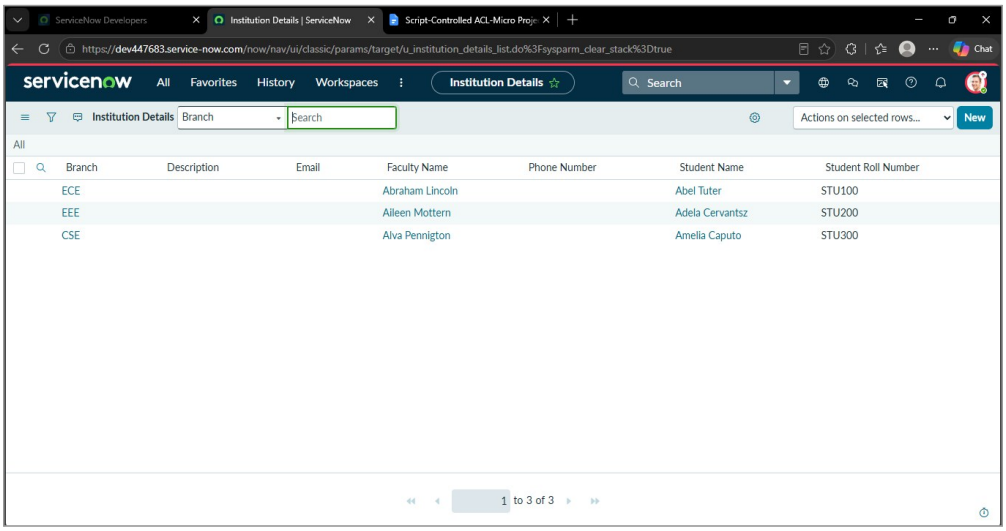


Figure 13: Administrator can view all records regardless of branch

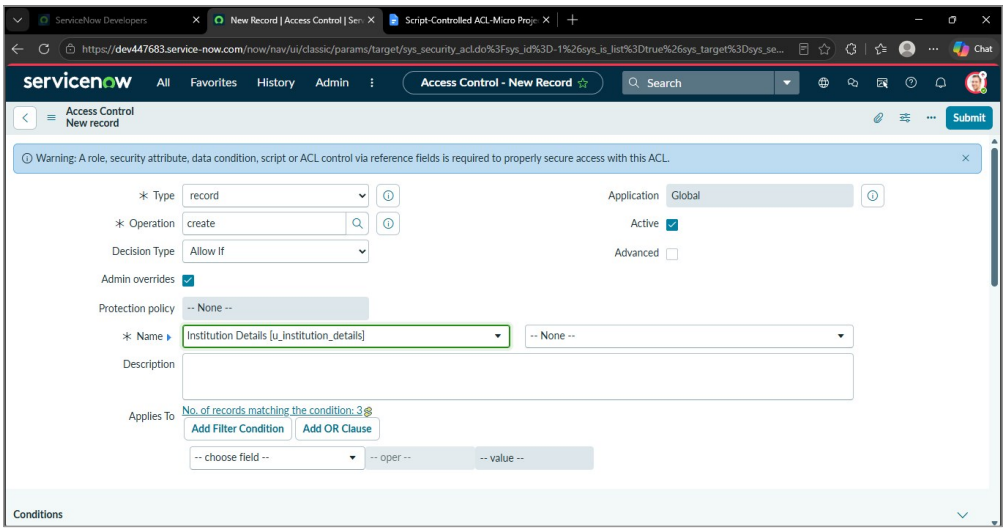


Figure 14: Create ACL – Operation create on u_institution_details

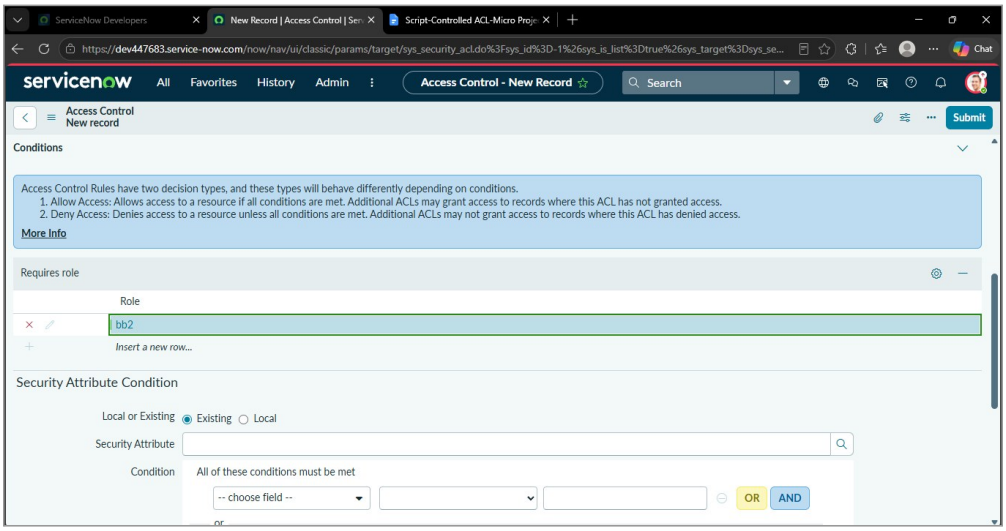


Figure 15: Create ACL – Requires role bb2

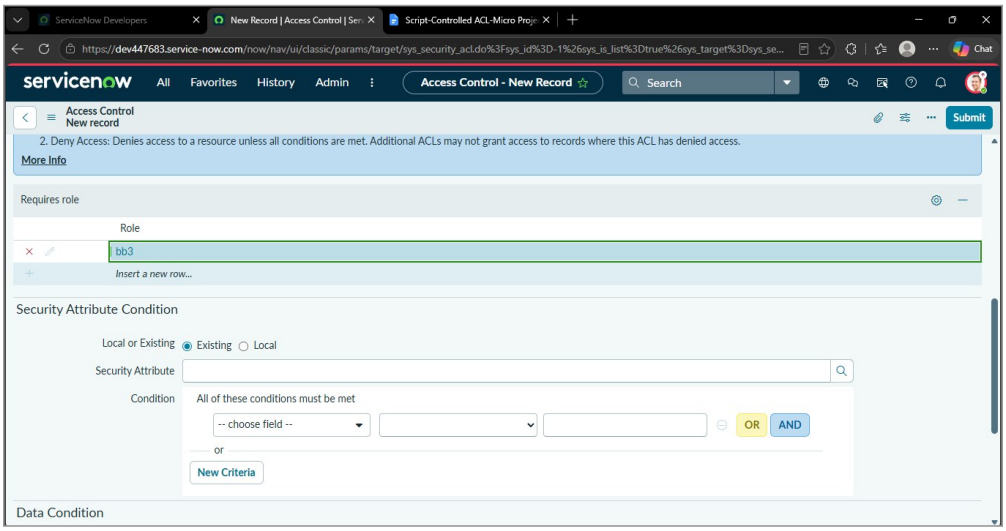


Figure 16: Write ACL – Requires role bb3

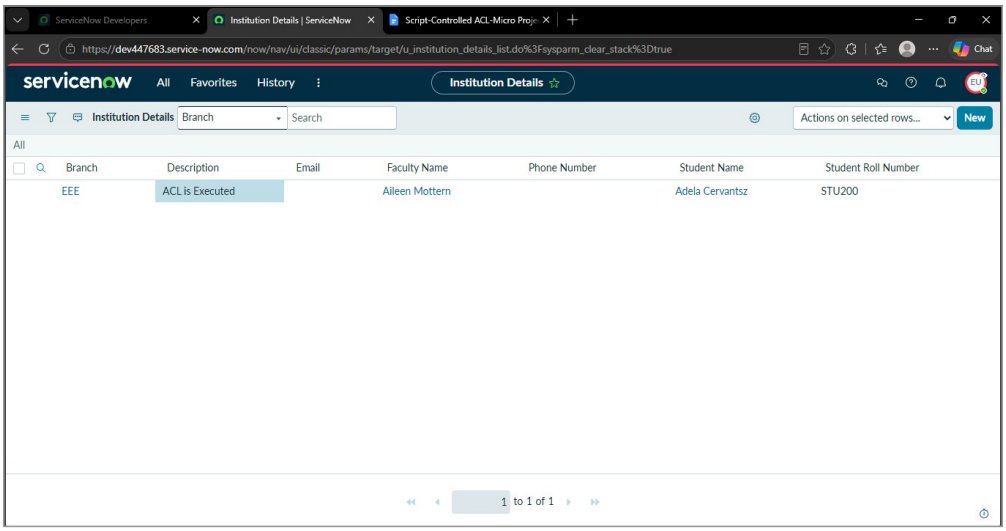


Figure 17: Record edited successfully by a user with write access (Description: ACL is Executed)

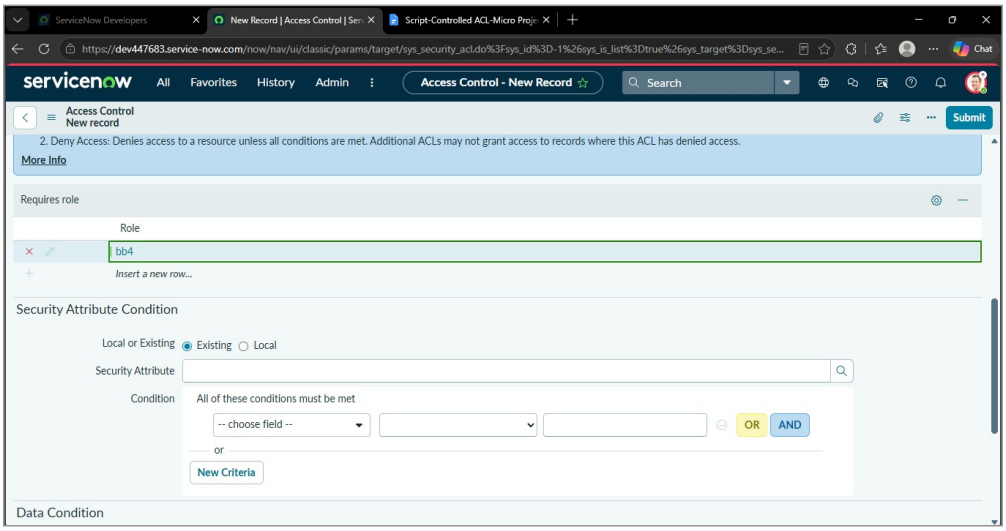


Figure 18: Delete ACL – Requires role bb4

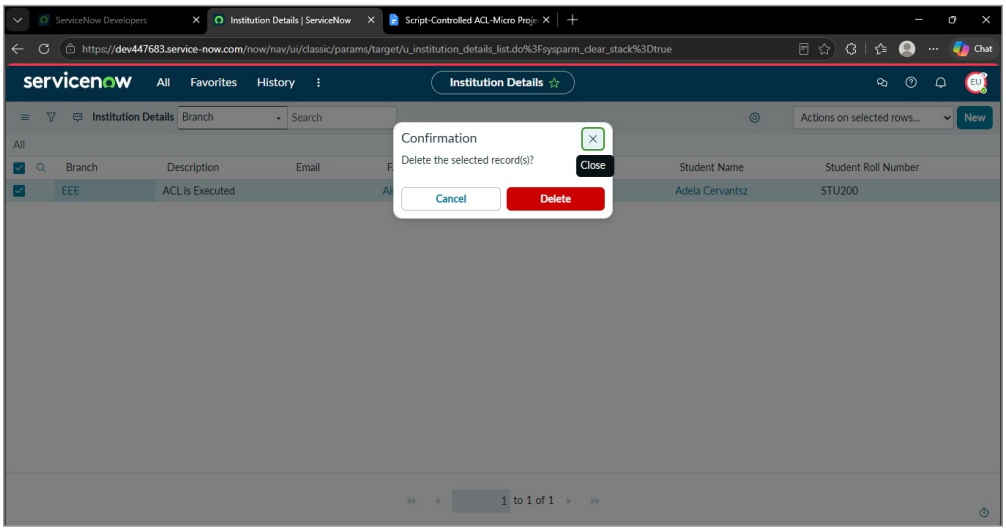


Figure 19: Delete confirmation for the selected EEE record

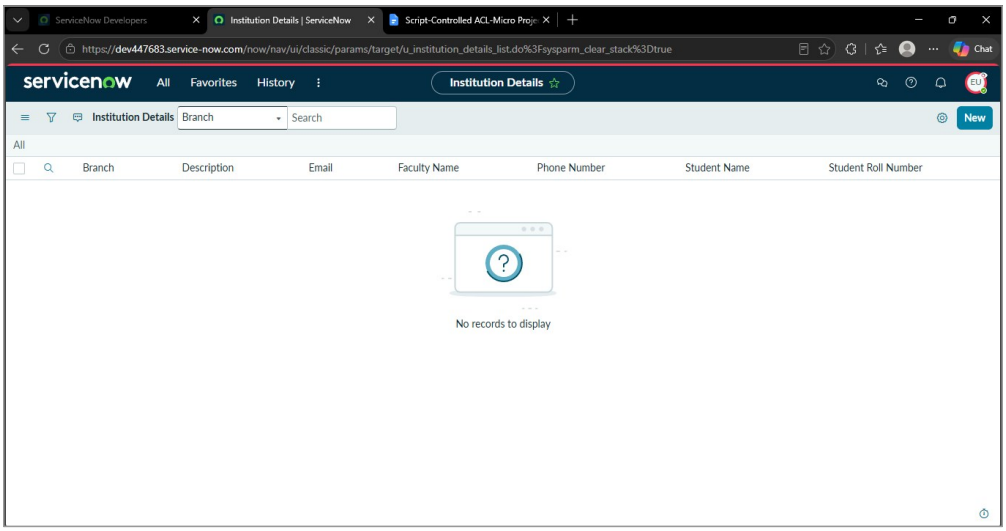


Figure 20: List after deletion – record removed

Demo video link: (to be added by the team)

10. Known Issues

No defect was recorded in the documented execution path. The ACL tests covered the positive and the no-role (denied) scenarios; a quantitative load test was not included in the available implementation evidence.

11. Future Enhancements

- Extend the ACLs to the ECE and CSE branches with dedicated roles and data conditions.
- Add field-level ACLs to protect sensitive fields such as Email and Phone Number.
- Use groups instead of individual roles to simplify access administration.
- Apply the same ACL pattern to Service Portal widgets and additional custom tables.

- Introduce controlled performance benchmarking and ACL debugging reports.

12. Appendix

Project resources:

- ServiceNow Platform: https://www.servicenow.com/
- ServiceNow Developer Portal: https://developer.servicenow.com/
- ServiceNow Access Control (ACL) Documentation: https://www.servicenow.com/docs/
- GitHub Repository: (to be added by the team)
- Demo Video: (to be added by the team)